Project Report (TensorFlow, Keras, Keras_CNN)

Dataset: - A_Z Handwritten Data

Introduction- The objective of this assignment is to build and compare three deep learning models for handwritten alphabet recognition using the A–Z Handwritten Alphabet dataset. The models include a TensorFlow Artificial Neural Network (ANN), a Keras Artificial Neural Network (ANN), and a Keras Convolutional Neural Network (CNN). Their performance was evaluated using test accuracy, classification reports, and confusion matrices.

TensorFlow ANN Model

Data Preprocessing:

I have loaded A–Z Handwritten Alphabet dataset CSV file using the Pandas library. Separated first column, containing the alphabet labels (0–25), from the remaining 784-pixel values representing each handwritten image. Converted image data to the **float32** data type and normalized by dividing the pixel values by **255**, scaling them to the range of 0–1. Then I splitted dataset into **80% training** and **20% testing** sets using train_test_split(). Since the TensorFlow ANN accepts one-dimensional input, each image was represented as a flattened vector of **784 input features**. Integer class labels were retained because I used Sparse **Categorical Crossentropy** loss function during training in the model.

```
# A_Z Handwritten dataset parameters
num_classes = 26 # total classes (26 alphabets)
num_features = 784 # data features (img shape: 28*28)

# Loading kaggle A_Z Handwritten dataset csv file.
# Updating the path of 'A_Z Handwritten Data.csv' file is located
csv_path = "A_Z Handwritten Data.csv"
dataset = pd.read_csv(csv_path).astype('float32')

# Extracting labels and features
# The first column contains the labels (0 to 25); the remaining 784 columns are pixels
x = dataset.iloc[:,1:].values
y = dataset.iloc[:,0].values

# Splitting dataset into Training(80%) & Testing(20%)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.20, random_state = 42)

# Normalizing pixel values
x_train = x_train / 255.0
x_test = x_test / 255.0

# Ensure data types are float32 and int64 respectively
y_train, y_test = y_train.astype(np.int64), y_test.astype(np.int64)
```

Model Architecture:

This model consists of **784 input neurons**, corresponding to the 784 pixels of each handwritten image, followed by **two hidden layers** containing **ReLU activation function.** The **output layer** contains **26 neurons**, and uses the **SoftMax activation function** for multi-class classification. The model is trained using the **Adam optimizer** with the **Sparse Categorical Crossentropy loss function**.

- Input layer: 784 neurons
- Hidden layer 1: 512 neurons (ReLU)
- Hidden layer 2: 256 neurons (ReLU)
- Output layer: 26 neurons (Softmax)
- Optimizer: Adam
- Loss Function: Sparse Categorical Crossentropy

Training & Hyperparameters:

- Learning rate: 0.001
- Batch size: 128
- Training steps: 5000

- Optimizer: Adam

Result Evaluation:

- Test Accuracy is **98.06%**

Evaluating the Trained Model

```
# Test model on validation set.
pred = neural_net(x_test)
print("Test Accuracy: %f" % accuracy(pred, y_test))
```

```
Test Accuracy: 0.980615
```

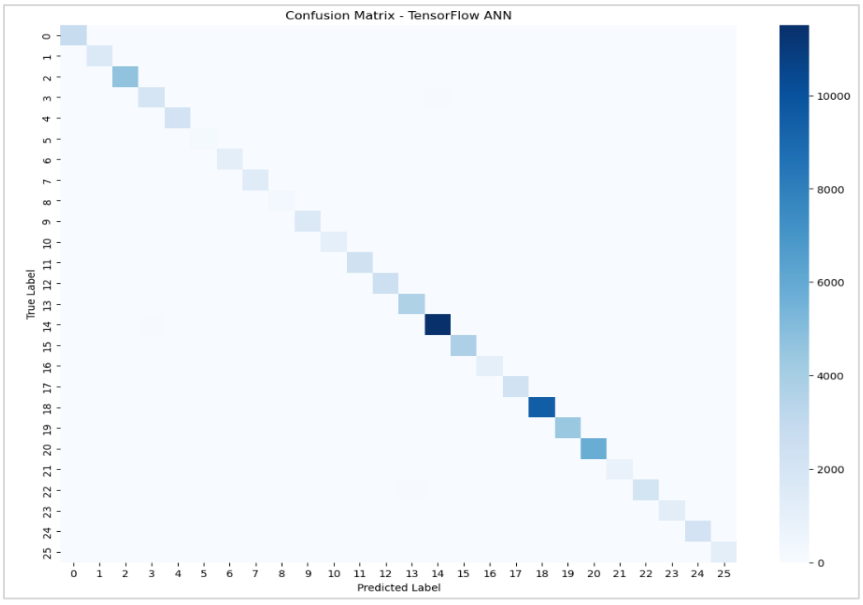
- **Classification Report:**

```
Classification Report:
              precision    recall  f1-score   support

           0       0.97      0.98      0.98      2806
           1       0.96      0.98      0.97      1673
           2       0.98      0.99      0.99      4742
           3       0.94      0.96      0.95      2044
           4       0.99      0.96      0.98      2214
           5       0.97      0.97      0.97       231
           6       0.96      0.95      0.96      1183
           7       0.94      0.95      0.95      1466
           8       0.97      0.98      0.97       237
           9       0.95      0.98      0.97      1668
          10       0.97      0.94      0.96      1132
          11       0.99      0.98      0.98      2319
          12       0.98      0.98      0.98      2487
          13       0.97      0.98      0.97      3756
          14       0.99      0.99      0.99     11629
          15       0.99      0.98      0.98      3868
          16       0.95      0.96      0.96      1159
          17       0.97      0.97      0.97      2296
          18       1.00      0.99      0.99      9556
          19       0.99      1.00      0.99      4447
          20       0.98      0.99      0.99      5860
          21       0.98      0.99      0.98       823
          22       0.99      0.96      0.97      2198
          23       0.98      0.97      0.98      1269
          24       0.99      0.96      0.97      2202
          25       0.98      0.99      0.98      1225

    accuracy                           0.98     74490
   macro avg       0.97      0.97      0.97     74490
weighted avg       0.98      0.98      0.98     74490
```

- **Confusion Matrix:**



Findings: TensorFlow ANN model achieved approximately **98.06%** test accuracy. The classification report showed **high precision, recall, and F1-scores** across most classes. The confusion matrix indicated only a small number of misclassifications.

Keras ANN Model

Data Preprocessing-

I have loaded A–Z Handwritten Alphabet dataset using the Pandas library and divided into image features and class labels. Then converted image data to the **float32** data type and normalized by

dividing each pixel value by **255**, resulting in values between 0 and 1. Then, Splitted dataset into **80% training** and **20% testing** sets using train_test_split(). Each handwritten image was flattened into a **784-dimensional feature vector**, making it suitable for the Artificial Neural Network. Converted class labels into **one-hot encoded vectors** containing **26 classes (A–Z)** to support multi-class classification using the **Categorical Crossentropy** loss function.

Loading Dataset

```
data = pd.read_csv("A_Z Handwritten Data.csv") # Loading A-Z Handwritten dataset from the CSV file
x = data.iloc[:,1:].values # Extracting the image pixel values (784 features)
y = data.iloc[:,0].values # Extracting the corresponding alphabet labels (0-25)
```

Data Preprocessing

```
x = x.astype("float32") # Converting the image pixel values to floating-point format
x = x/255.0 # Normalizing the pixel values to the range 0-1

# Splitting the dataset into training and testing sets (80% training, 20% testing)
train_images,test_images,train_labels,test_labels = train_test_split(x,y,test_size=0.20,random_state=42,stratify=y)
```

One-Hot Encoding the Labels

```
train_labels = keras.utils.to_categorical(train_labels, 26) # Converting the training labels into one-hot encoded vectors (26 alphabet classes)
test_labels = keras.utils.to_categorical(test_labels, 26) # Converting the testing labels into one-hot encoded vectors (26 alphabet classes)
```

Model Architecture:

This Model consists of an **input layer with 784 neurons**, corresponding to the 784 pixels of each handwritten image, followed by **one hidden layer** with 512 neurons using the **ReLU activation function**. A **Dropout layer** (30%) is included to reduce overfitting during training. The **output layer** contains **26 neurons**, representing the 26 alphabet classes (A–Z), and uses the **Softmax activation function** for multi-class classification. The model is trained using the **RMSprop optimizer** with the **Categorical Crossentropy loss function**.

- Input: 784
- Hidden Layer: 512 neurons (ReLU)
- Dropout
- Output: 26 classes (Softmax)
- Optimizer: RMSprop

Training & Hyperparameters: -

- Batch size: 128
- Epochs: 15
- Validation split: 10%
- Optimizer: RMSprop

Result Evaluation: -

- Test Accuracy is **98.58%**

Evaluating the Trained Keras ANN Model

```
score = model.evaluate(test_images, test_labels, verbose=0) # Evaluating the trained model using the testing dataset
print('Test loss:', score[0]) # Display the test loss
print('Test accuracy:', score[1]) # Display the test accuracy
```

```
Test loss: 0.06073954701423645
Test accuracy: 0.98585045337677
```

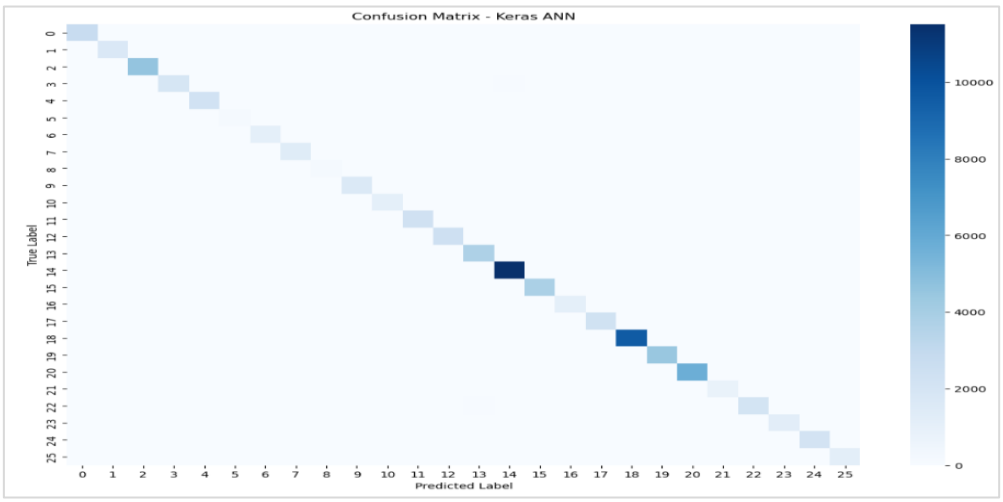
- **Classification Report:**

```
2328/2328 ━━━━━━━━━━━━━━━━━━━━ 11s 5ms/step
Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.99      0.98      2774
           1       0.98      0.98      0.98      1734
           2       0.98      0.99      0.99      4682
           3       0.98      0.95      0.97      2027
           4       0.99      0.97      0.98      2288
           5       0.96      0.94      0.95       233
           6       0.99      0.97      0.98      1152
           7       0.97      0.96      0.97      1444
           8       0.97      0.96      0.97       224
           9       0.98      0.97      0.98      1699
          10       0.97      0.97      0.97      1121
          11       0.99      0.99      0.99      2317
          12       0.99      0.98      0.99      2467
          13       0.97      0.99      0.98      3802
          14       0.99      1.00      0.99     11565
          15       0.98      0.99      0.99      3868
          16       0.98      0.95      0.97      1162
          17       0.99      0.98      0.98      2313
          18       1.00      0.99      0.99      9684
          19       0.99      0.99      0.99      4499
          20       0.99      0.99      0.99      5801
          21       0.98      0.99      0.98       836
          22       0.99      0.96      0.98      2157
          23       0.97      0.98      0.98      1254
          24       0.98      0.98      0.98      2172
          25       0.98      0.99      0.99      1215

    accuracy                           0.99     74490
   macro avg       0.98      0.98      0.98     74490
weighted avg       0.99      0.99      0.99     74490
```

- **Confusion Matrix:**



Findings: Keras ANN achieved approximately 98.58% test accuracy. The evaluation metrics and confusion matrix demonstrated strong and consistent classification performance across the alphabet classes.

Keras CNN Model

Data Preprocessing:

Loaded A–Z Handwritten Alphabet dataset from a CSV file using Pandas, with the first column containing the alphabet labels and the remaining columns representing the pixel values of each handwritten image. Converted image data to the **float32** data type and normalized by dividing the pixel values by **255**, scaling them to the range of 0–1. After splitting the dataset into **80% training** and **20% testing** sets, each image was reshaped into **28 × 28 × 1** format to preserve its spatial information for the Convolutional Neural Network. Finally, converted class labels into **one-hot encoded vectors** with **26 output classes**, enabling multi-class classification using the **Categorical Crossentropy** loss function.

Loading and Splitting the Dataset

```
# Load the A-Z Handwritten Alphabet dataset and convert values to float32
dataset = pd.read_csv('A_Z Handwritten Data.csv', header=None).astype('float32')

# The first column contains the labels (0-25)
alphabet_labels = dataset.iloc[:, 0].values
# The remaining 784 columns are the pixel values
alphabet_images = dataset.iloc[:, 1:].values

# splitting data into train and test sets
train_images, test_images, train_labels_raw, test_labels_raw = train_test_split(alphabet_images, alphabet_labels, test_size=0.2, random_state=42)
```

Reshaping and Normalizing the Image Data

```
from tensorflow.keras import backend as K # Import the Keras backend module

# Check the image data format required by the Keras backend
if K.image_data_format() == 'channels_first':
    # Reshape images to (samples, channels, height, width)
    train_images = train_images.reshape(train_images.shape[0], 1, 28, 28)
    test_images = test_images.reshape(test_images.shape[0], 1, 28, 28)
    # Store the input shape for the CNN model
    input_shape = (1, 28, 28)
else:
    # Reshape images to (samples, height, width, channels)
    train_images = train_images.reshape(train_images.shape[0], 28, 28, 1)
    test_images = test_images.reshape(test_images.shape[0], 28, 28, 1)
    # Store the input shape for the CNN model
    input_shape = (28, 28, 1)

# Convert image data to floating-point format
train_images = train_images.astype('float32')
test_images = test_images.astype('float32')
# Normalize pixel values to the range 0-1
train_images /= 255
test_images /= 255
```

One-Hot Encoding the Labels

```
# Converting the training & Testing labels into one-hot encoded vectors (26 alphabet classes)
train_labels = tensorflow.keras.utils.to_categorical(train_labels_raw, 26)
test_labels = tensorflow.keras.utils.to_categorical(test_labels_raw, 26)
```

Model Architecture:

This Model consists of two convolutional layers with 32 and 64 filters, respectively, both using the ReLU activation function to extract spatial features from handwritten alphabet images. These are followed by a Max Pooling layer to reduce the feature map dimensions and a Dropout layer (25%) to minimize overfitting. The extracted features are then flattened and passed through a fully connected dense layer with 128 neurons using ReLU activation, followed by another Dropout layer (50%). Finally, the output layer contains 26 neurons with the Softmax activation function to classify the handwritten images into the 26 alphabet classes (A–Z). The model is trained using the Adam optimizer with the Categorical Crossentropy loss function.

- Conv2D (32 filters)
- Conv2D (64 filters)
- MaxPooling2D
- Dropout (0.25)
- Flatten
- Dense (128, ReLU)
- Dropout (0.5)
- Dense (26, Softmax)
- Optimizer: Adam
- Loss function: Categorical Crossentropy

Training & Hyperparameters:

- Batch size: 50

- Epochs: 5
- Optimizer: Adam

Result Evaluation:

- Test Accuracy is **98.95%**

Evaluating the Trained CNN Model

```
# Evaluate the trained CNN model using the testing dataset
score = model.evaluate(test_images, test_labels, verbose=0)
print('Test loss:', score[0]) # Display the test loss
print('Test accuracy:', score[1]) # Display the test accuracy
```

```
Test loss: 0.03797617554664612
Test accuracy: 0.9895557761192322
```

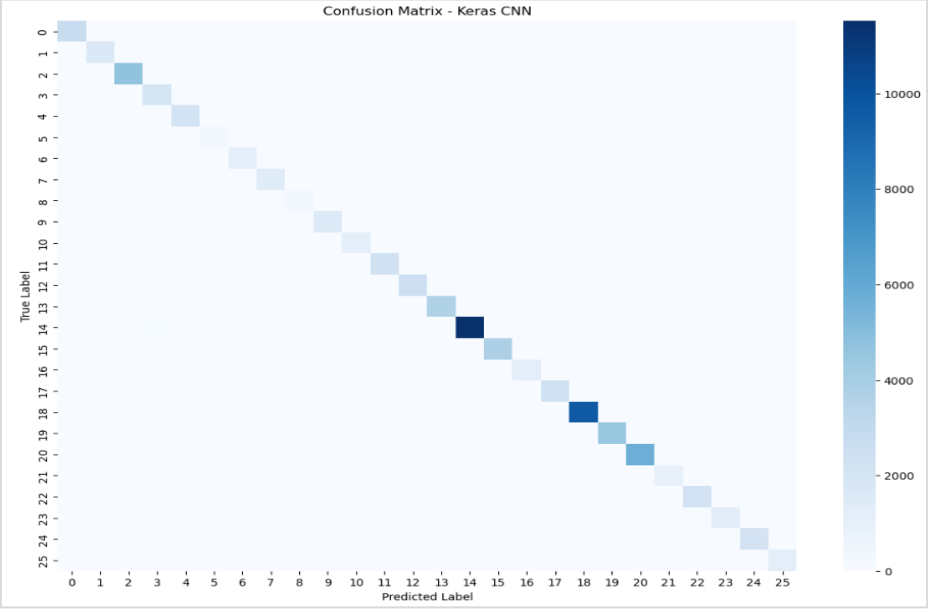
- **Classification Report:**

```
Classification Report:
              precision    recall  f1-score   support

           0       0.98      1.00      0.99      2806
           1       0.99      0.99      0.99      1673
           2       0.99      0.99      0.99      4742
           3       0.94      0.98      0.96      2044
           4       1.00      0.99      0.99      2214
           5       0.99      0.98      0.99       232
           6       0.99      0.97      0.98      1183
           7       0.97      0.99      0.98      1456
           8       0.98      0.99      0.98       242
           9       0.98      0.98      0.98      1655
          10       0.98      0.98      0.98      1113
          11       0.99      0.99      0.99      2306
          12       1.00      0.99      0.99      2502
          13       0.99      0.99      0.99      3700
          14       1.00      0.99      0.99     11675
          15       0.99      0.99      0.99      3840
          16       0.99      0.98      0.99      1184
          17       0.99      0.99      0.99      2306
          18       1.00      1.00      1.00      9663
          19       0.99      1.00      0.99      4498
          20       0.99      1.00      0.99      5774
          21       0.99      1.00      0.99       863
          22       1.00      0.98      0.99      2177
          23       0.99      0.98      0.98      1259
          24       0.98      0.98      0.98      2172
          25       1.00      0.99      1.00      1212

    accuracy                           0.99     74491
   macro avg       0.99      0.99      0.99     74491
weighted avg       0.99      0.99      0.99     74491
```

- **Confusion Matrix:**



Findings: Keras CNN achieved the **highest performance** with approximately **99.00% test accuracy**. The **classification** report and **confusion matrix** confirmed excellent classification accuracy with very few misclassifications.

Comparison of Models:

Model	Test Accuracy	Observation
TensorFlow ANN	**98.06%**	**Good baseline model with high accuracy**
Keras ANN	**98.58%**	**Improved accuracy with simpler implementation**
Keras CNN	**98.95%**	**Highest accuracy and best overall performance**

Discussion:

Challenges:
Selecting appropriate model architecture and training parameter (e.g.- no. of hidden layers, training epochs, batch size along with correct optimizer) while ensuring that model should give good performance without overfitting or underfitting.

Limitations:
Since I have trained & evaluated all three models using A_Z Handwritten Dataset so model performance may vary for different dataset or unseen dataset.

Potential Improvements:
For further improvement in classification performance and model generalization, other architectures like batch normalization, early stopping or learning rate scheduling can be use.

Conclusion:
Three deep learning models were implemented and evaluated using the A–Z Handwritten Alphabet dataset. The TensorFlow ANN and Keras ANN achieved high classification accuracy, while the Keras CNN provided the best overall performance with approximately 99% test accuracy. The evaluation metrics and confusion matrices demonstrate that CNNs are more effective for handwritten image classification because they automatically learn important spatial features from the images.

References

1. TensorFlow Documentation. https://www.tensorflow.org/
2. Keras Documentation. https://keras.io/
3. TensorFlow Keras API Documentation. https://www.tensorflow.org/api_docs/python/tf/keras